MACHINE LEARNING · DATA PREPARATION

QWIK GUIDE
ML FOUNDATIONS
DATA QUALITY

Handling Missing Values & Outliers in Data

A beginner's field guide to finding, understanding, and fixing the two most common data-quality problems — with plain-language definitions, worked intuition, and illustrations.

A Training Guide for the **AI Foundations Series**

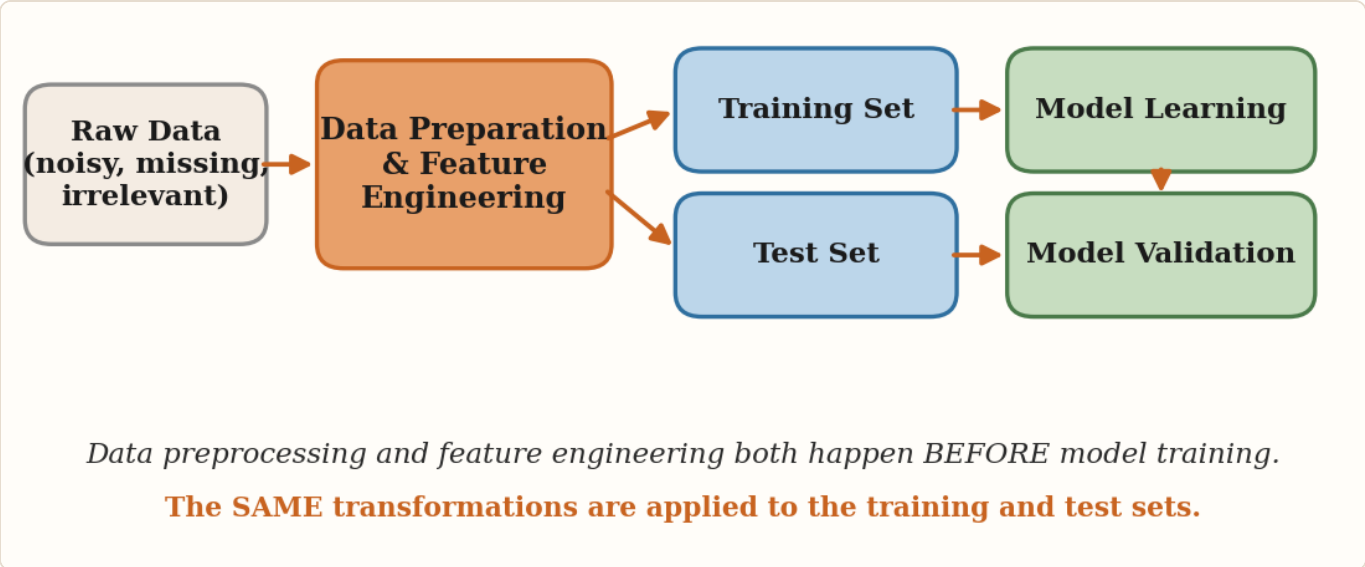
David Grunwald · 2026

Before You Begin

Real-world data is messy. Before a machine learning model ever sees it, someone has to **clean it up** — and two problems come up again and again: **missing values** (gaps where data should be) and **outliers** (values that sit far away from everything else). Handle them badly and even a great model will give you poor, biased answers. Handle them well and you build on solid ground.

This guide walks you through the full toolkit a beginner needs: how to spot these problems, the vocabulary to describe them precisely, and the standard techniques to fix them. Each idea is explained in everyday language first, then backed by a small illustration or code snippet. A **Quick Reference** table, a full **glossary**, and a curated **Learn More** list round things out.

> **THE ONE RULE TO REMEMBER**
>
> Always **split your data into training and test sets first**, then learn any cleaning rule (imputation values, outlier thresholds, scaling) from the **training set only**. Letting the test set influence those rules is called *data leakage* and it quietly inflates your results.



Where data preparation sits in the machine learning workflow. Cleaning happens before training, and the same transformations are applied to both the training and test sets.

1 Data Preparation: Preprocessing vs. Feature Engineering

Raw data is the unprocessed data collected from various sources. It usually contains noise, missing values, and information you don't need. **Data preparation** turns it into something a model can actually learn from, and it has two related-but-different halves.

Data preprocessing

Cleaning and transforming raw data into a usable format. It is *model-agnostic* — you'd do it no matter which model you plan to use. It includes handling missing values, scaling or normalizing numbers, encoding categories into numbers, and detecting outliers. Good preprocessing improves data quality, can reduce model complexity and overfitting, and lowers computational cost (and cloud bills).

Feature engineering

Creating or reshaping the input variables (called **features** or **X variables**) to help a *specific* model do better. Unlike preprocessing, this leans on **domain knowledge**. Examples: combining two columns into one, or taking the log of an exponential relationship so a simple model like linear regression can handle it.

> **BEGINNER TAKEAWAY**
>
> Both happen **before model training**. Preprocessing = "make the data clean and valid for any model." Feature engineering = "make the data *smart* for this model and this problem."

2 Missing Values: What They Are & Why They Matter

A **null value** (or missing value) represents the *absence* of data. It is a placeholder for missing, unknown, or not-relevant information. Important: a null is **not** the number zero, and it is **not** an empty string — it signals a complete lack of information.

Ignoring nulls is risky. They can **bias** your analysis (especially if a whole group is missing), confuse ML algorithms, distort statistics like the mean, and even increase computing cost. Occasionally the fact that a value is missing is *itself* informative — for instance, a survey question a respondent chose not to answer.

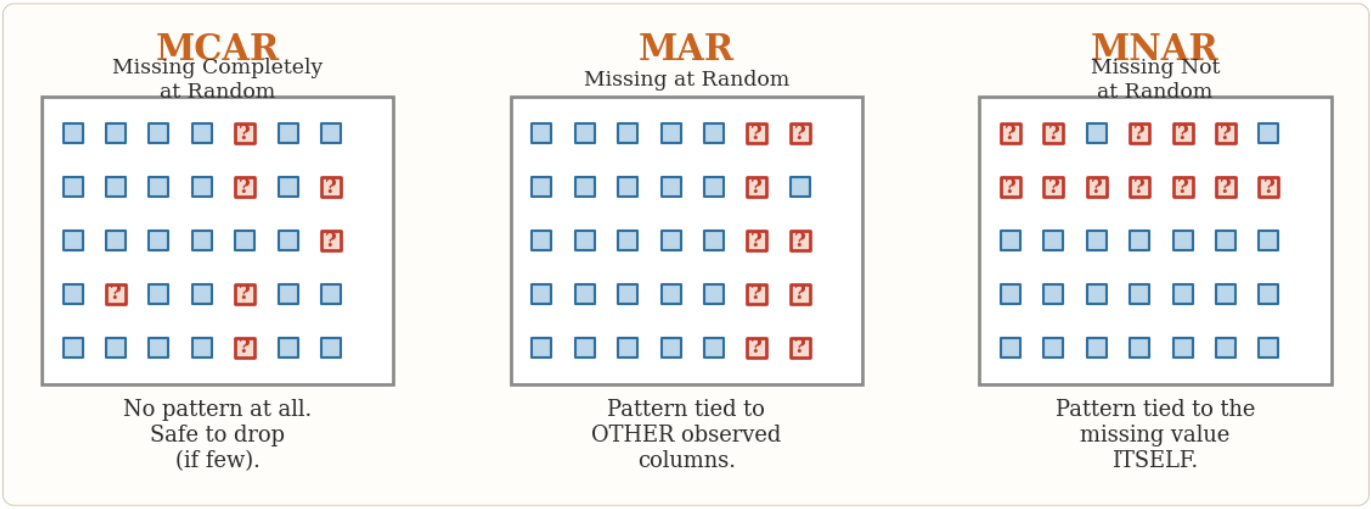
Types of missing values

Type	What it looks like
`Explicit null`	Formally marked as missing: `NaN`, `NULL`, or `None`.
`Implicit null`	Missing but not marked: an empty string, whitespace, or a placeholder like `NA` or `---`.
`System-generated`	Introduced by import, export, or storage errors.
`Out-of-bound`	Invalid values outside a sensible range (negative age, a future birth date).
`Contextual null`	Looks valid but is meaningless in context (a height or weight of 0).

3 Missing Data Mechanisms: MCAR, MAR & MNAR

Before choosing a fix, ask *why* the data is missing. There are three classic patterns, and they decide whether it's safe to simply delete rows.

Mechanism	Meaning	Plain-language example
MCAR	Missing Completely At Random — missingness is unrelated to any data, observed or not.	A sensor randomly drops a few readings.
MAR	Missing At Random — missingness relates to *other observed* columns, not the missing value itself.	Men skip an "emotions" question more often, but gender is recorded.
MNAR	Missing Not At Random — missingness depends on the *unobserved value itself*.	High earners decline to report income.



The three missing-data mechanisms. Red '?' squares are missing. Under MCAR they scatter randomly; under MAR they cluster by an observed column; under MNAR they cluster by the hidden value itself.

WHY THIS MATTERS

Deletion is safest under **MCAR**. Under **MAR** or **MNAR**, dropping rows can bias your dataset — and under MNAR even ordinary imputation can be unreliable unless the missingness is explicitly modeled.

A SUBTLE BUT IMPORTANT POINT

MCAR, MAR, and MNAR describe **assumptions about the process** that created the gaps — they generally can't be proven from the data alone. In particular, you can't confirm MNAR just by inspecting the values you *do* have. Lean on domain knowledge to reason about which mechanism is plausible.

4 Fixing Missing Data (1): Deletion

The simplest fix: remove the rows or columns that contain nulls. In pandas the workhorse is `dropna`.

```
# Drop any ROW that has a missing value
df_rows = df.dropna()

# Drop any COLUMN that has a missing value
df_cols = df.dropna(axis=1)
```

Deletion is reasonable when the proportion of nulls is **small** and the data is **MCAR**. It's straightforward and keeps only complete records.

> **THE CATCH**
>
> Deletion **shrinks your dataset** and throws away information. Worse, if the missingness is *systematic* (not random), the rows you drop may share a hidden pattern — introducing bias and distorting the data distribution. In one demo, dropping rows cut a dataset from 1,470 to 1,355 rows; dropping columns cut it from 34 to 25.

5 Fixing Missing Data (2): Simple Imputation

Imputation means replacing nulls with estimated values instead of deleting them — preserving your data. The most basic form is **univariate** imputation, which looks only at the column being filled. In scikit-learn this is the `SimpleImputer`.

```
from sklearn.impute import SimpleImputer

# Numeric columns: fill with the median (or 'mean')
imp = SimpleImputer(strategy='median')
X[num_cols] = imp.fit_transform(X[num_cols])

# Categorical columns: fill with the most common value
imp_cat = SimpleImputer(strategy='most_frequent')
X[cat_cols] = imp_cat.fit_transform(X[cat_cols])
```

Common single-column strategies:

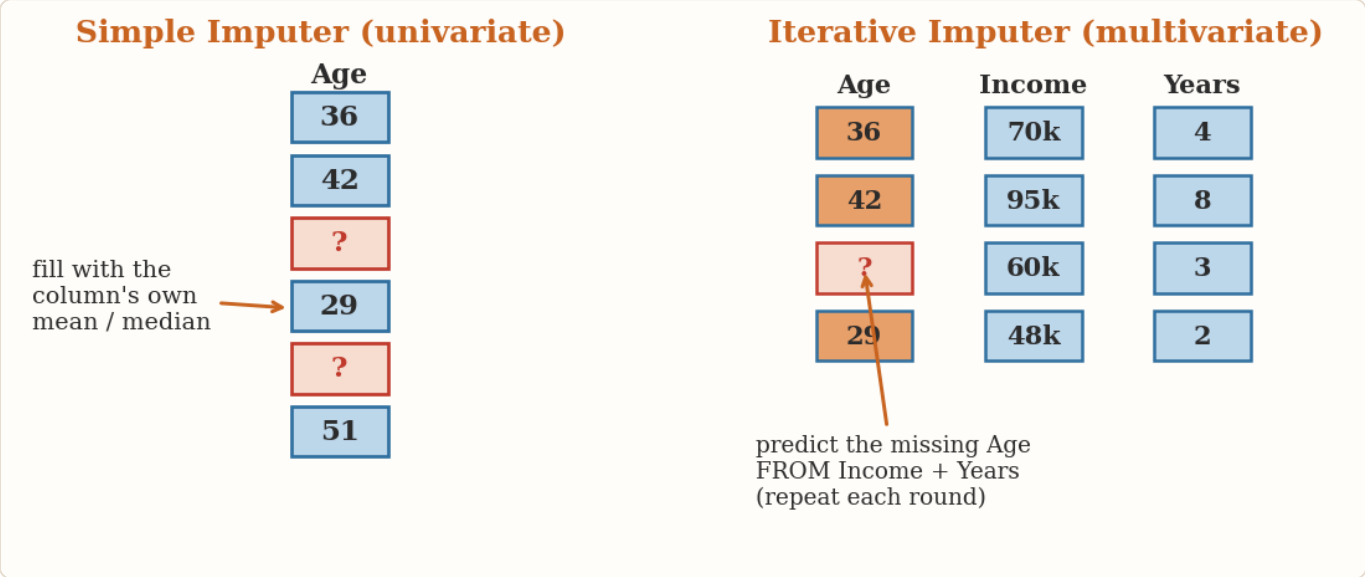
- **Mean / median / mode** — mean or median for numbers, mode (most frequent) for categories. Simple, but can distort the distribution if data isn't MCAR.
- **Forward fill / backward fill** — carry the previous (or next) observed value into the gap. Ideal for **time-series** data where nearby values are related.

- **Predictive imputation** — use a model such as K-nearest neighbors or regression to estimate the value from other features.

> **WATCH THE STATISTICS**
>
> If you impute with the **median**, the column's median won't change — but its **mean** will shift. That's expected: you're adding values, so averages move.

6 Fixing Missing Data (3): Multivariate Imputation

A smarter approach is **multivariate** imputation, which fills a column using information from the *other* columns. Scikit-learn's `IterativeImputer` does this in a round-robin: it treats each incomplete column as the target of a regression, predicts its missing values from the rest, and repeats for several rounds until the estimates settle.



Simple vs. iterative imputation. The Simple Imputer fills a gap from the column's own average; the Iterative Imputer predicts it from the other columns (here, Age is estimated from Income and Years).

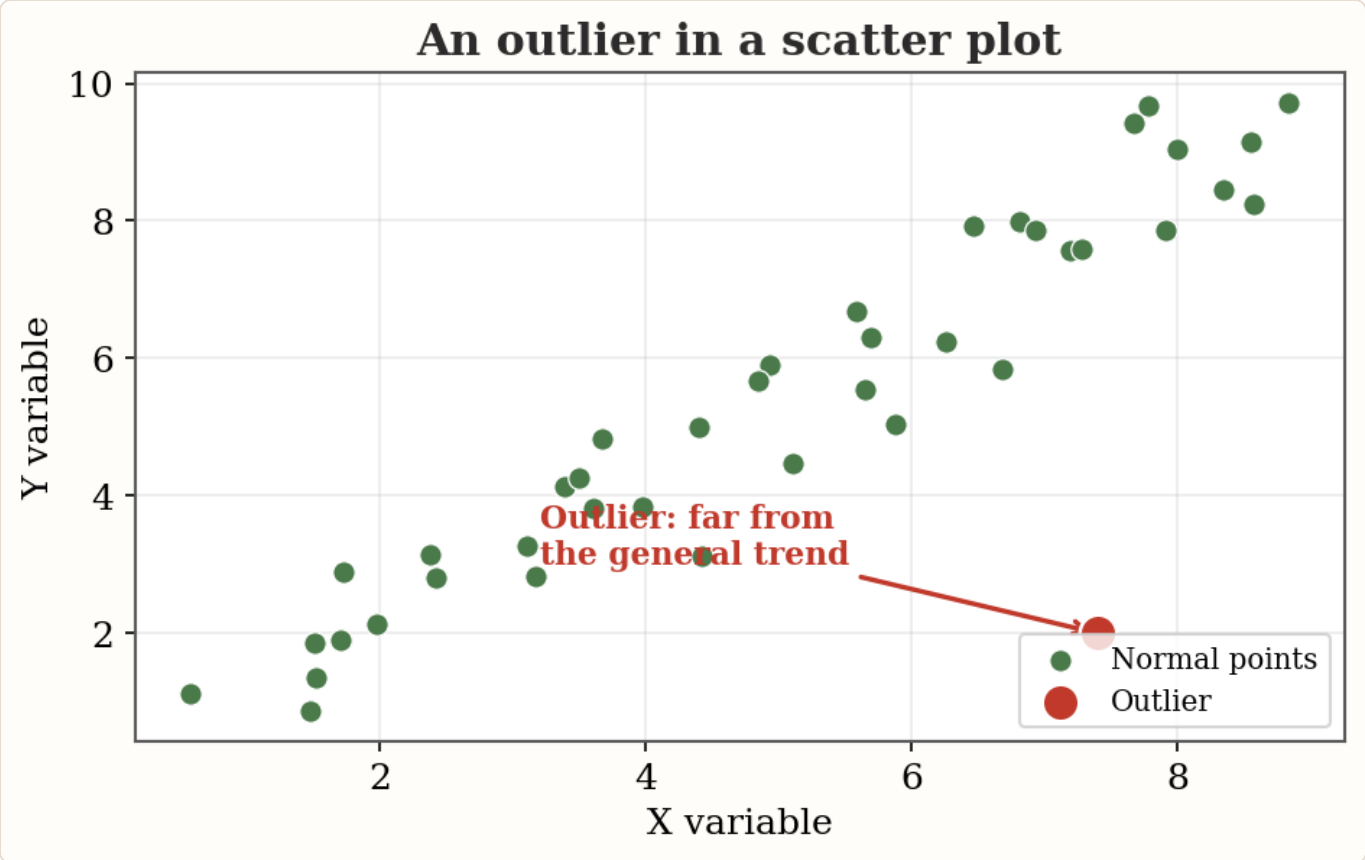
```
# IterativeImputer is still experimental, so enable it first
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imp = IterativeImputer()
X[num_cols] = imp.fit_transform(X[num_cols])
```

> **FIT VS. TRANSFORM**
>
> Across scikit-learn, `fit` *learns* from data and `transform` *applies* what was learned. `fit_transform` does both in one step. Think of fit as "train" and transform as "apply."

7 Outliers: What They Are & Their Impact

An **outlier** is a data point that deviates significantly from the typical range of a dataset. It might be a genuine rare event, or it might be an error — from a mistyped entry, a broken sensor, or a measurement mistake. (The terms **outlier** and **anomaly** are often used interchangeably; this guide uses *outlier* throughout.) There's no single hard definition, but there are clear *types*:

- **Global outlier** — extreme versus the whole dataset (if adult giraffes average about 16 feet, a 9-foot giraffe stands far outside the normal range).
- **Contextual outlier** — unusual only in a context (80°F is normal in summer, an outlier in winter).
- **Collective outlier** — a *group* that's abnormal together, even if each point looks fine alone (a week-long winter warm streak).



A single outlier (red) sitting far from the linear trend of the normal points (green). Even one such point can distort a model.

WHY OUTLIERS ARE DANGEROUS

They reduce the power of statistical tests, inflate error variance, and **distort regression** by pulling the slope and intercept — which throws off *every* prediction. They can bias ML models and hurt stability and interpretability. (That said, not every outlier is an error; some carry real signal, so investigate before deleting.)

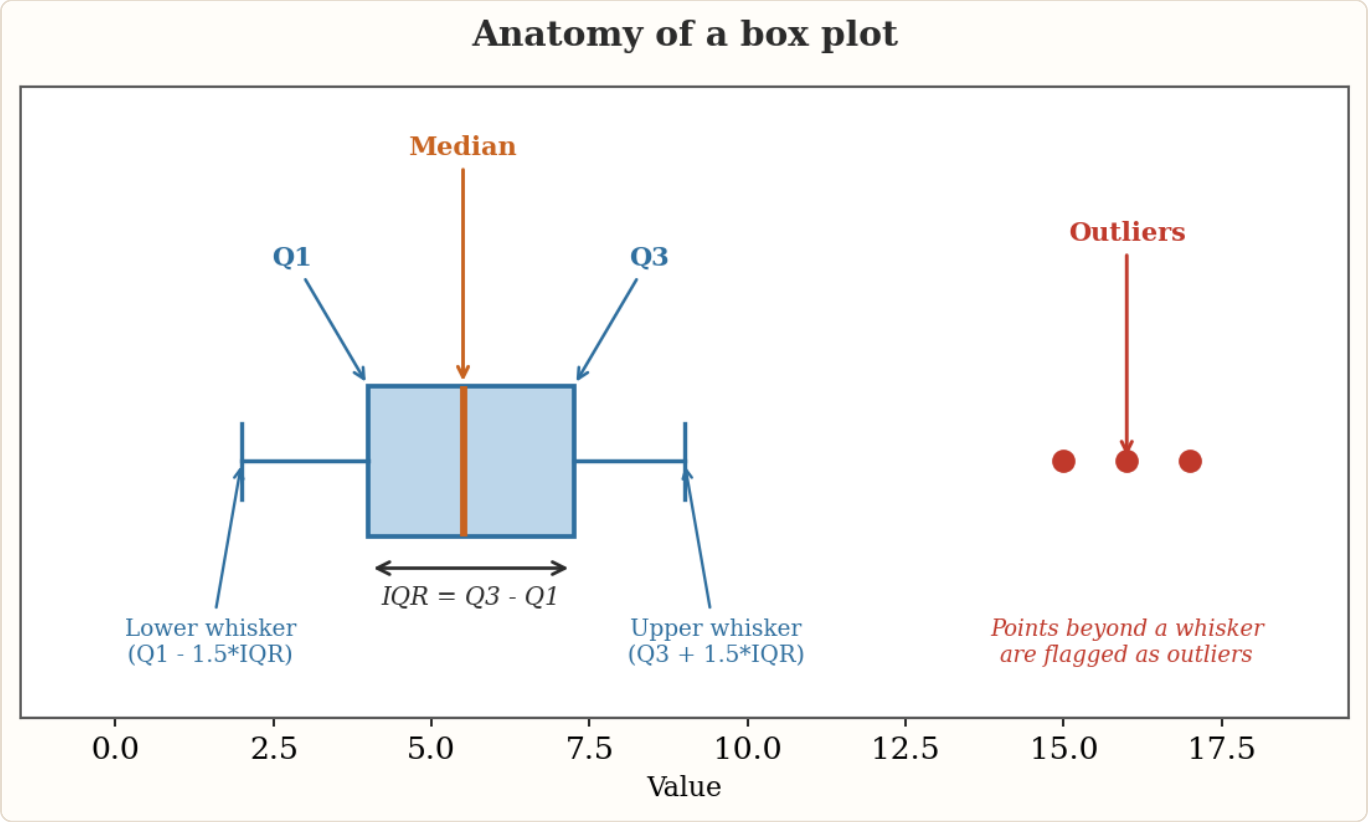
8 Detecting Outliers (1): Visualization

The fastest way to spot outliers is to *look* at the data. Three plots do most of the work:

- **Scatter plot** — reveals points that fall outside the general trend between two variables.
- **Histogram** — outliers appear as isolated bars far from the main cluster.
- **Box plot** — the go-to summary: it draws the median, quartiles, and whiskers, and plots outliers as individual dots.

Reading a box plot

The box spans **Q1** (25th percentile) to **Q3** (75th percentile). The line inside is the **median**. The whiskers reach out to the most extreme points still within **1.5 × IQR** of the box. Anything past a whisker is drawn as an outlier.



The parts of a box plot. The box is the middle 50% of the data (Q1 to Q3); whiskers extend 1.5xIQR beyond it; the three red dots past the upper whisker are flagged as outliers.

```
# Quick box plots with matplotlib / seaborn
import seaborn as sns
sns.boxplot(y=df['MonthlyIncome'])
```

9 Detecting Outliers (2): The IQR Method

The box plot's rule can be computed directly. The **interquartile range** is the spread of the middle 50% of the data:

```
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Anything below lower_bound or above upper_bound is an outlier
outliers = (data < lower_bound) | (data > upper_bound)
```

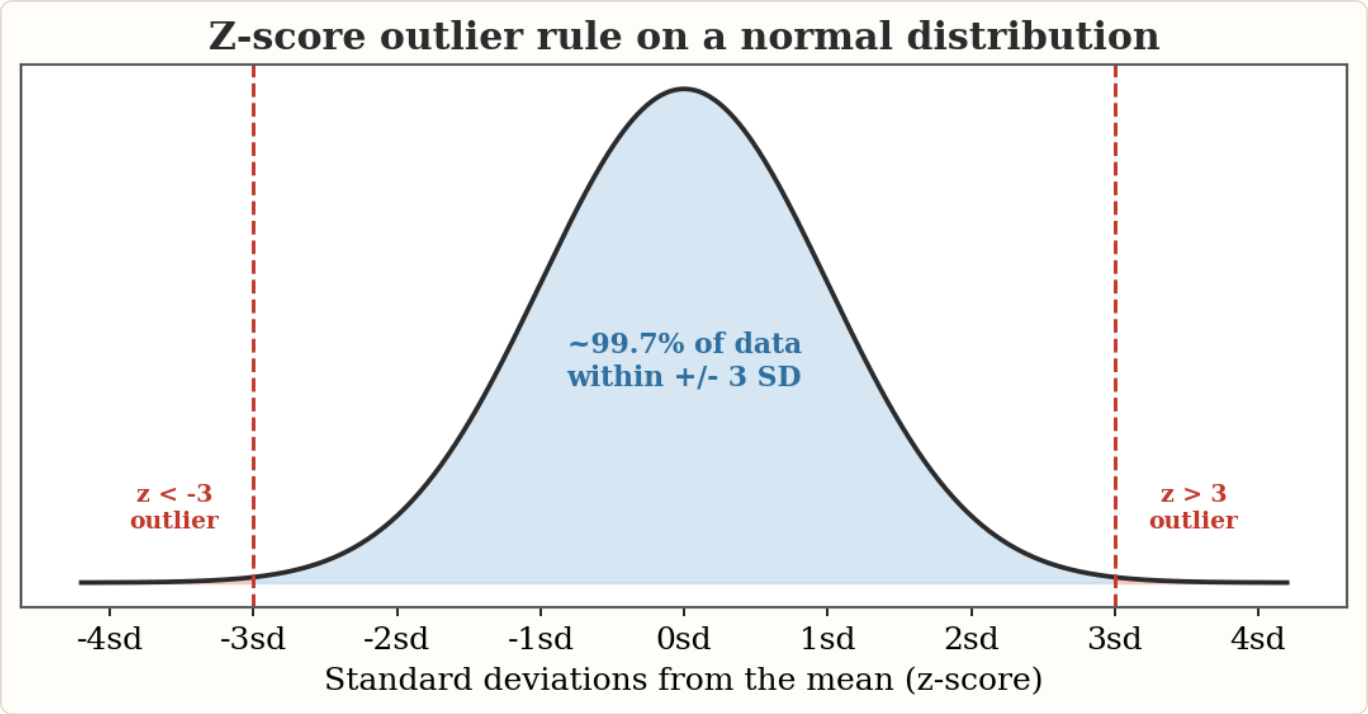
The IQR method is **distribution-free** — it doesn't assume your data is bell-shaped, which makes it a safe default. Because it's based on quartiles rather than the mean, it isn't dragged around by the very extreme values it's trying to find.

> **COMPUTE BOUNDS ON THE FULL COLUMN**
>
> When capping, calculate Q1, Q3, and IQR from the **entire** column (outliers included), not just the "clean" points. This gives consistent, unbiased thresholds.

10 Detecting Outliers (3): The Z-Score Method

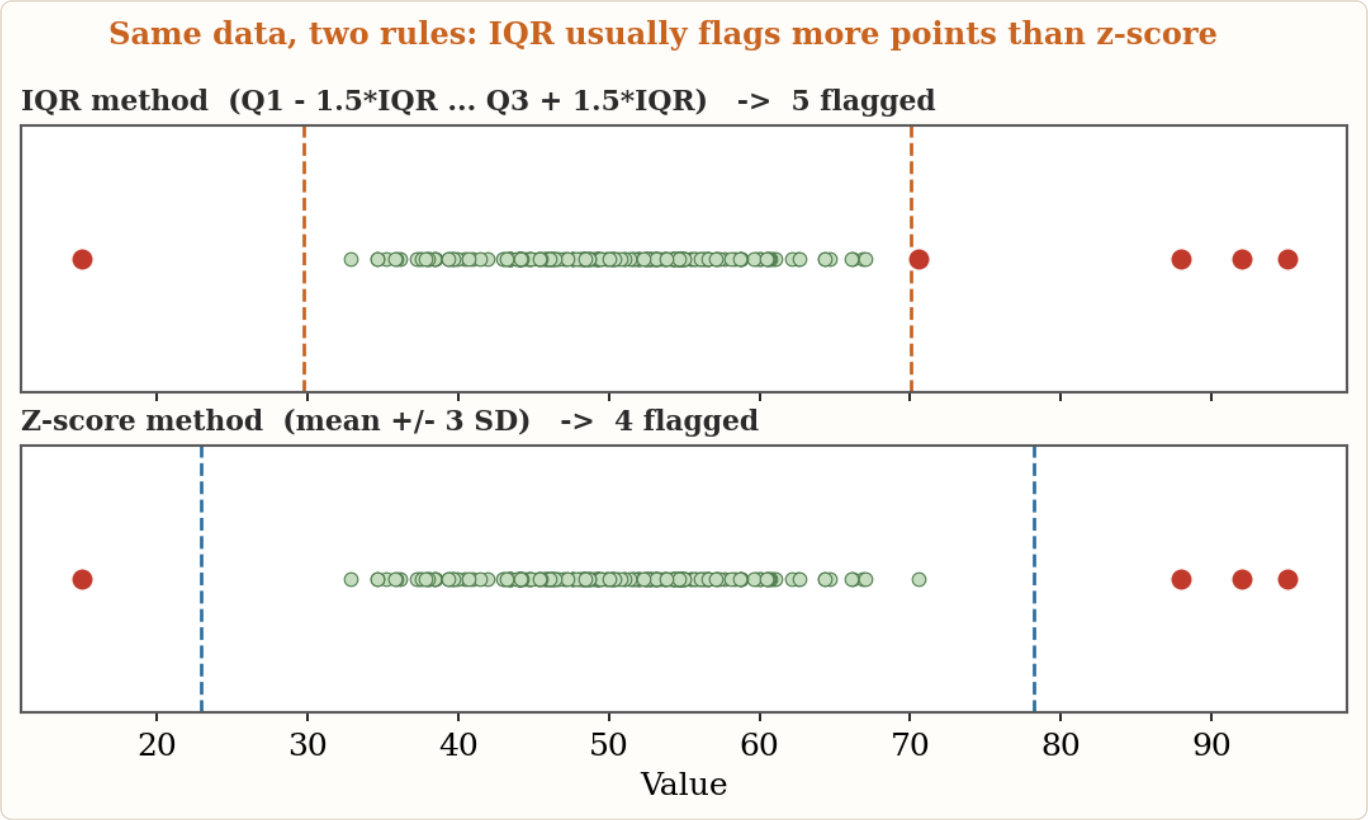
The **z-score** measures how many standard deviations a point sits from the mean. A point 3 SD above the mean has z = 3; one 3 SD below has z = -3. The common rule flags any point whose **absolute z-score exceeds 3** — a widely used rule of thumb, not a universal cutoff.

```
z = (value - mean) / std_dev

# Flag points more than 3 SD from the mean (either side)
outliers = np.abs(z) > 3
```



The z-score rule on a normal distribution. About 99.7% of values fall within +/-3 SD of the mean, so points beyond that (red tails) are rare enough to flag as outliers.

Because roughly **99.7%** of a normal distribution lies within ±3 SD, the z-score method works beautifully for **normally distributed** data and for very large datasets. It is a poor choice for **fat-tailed** or skewed data, where the mean and standard deviation are themselves distorted by extremes.



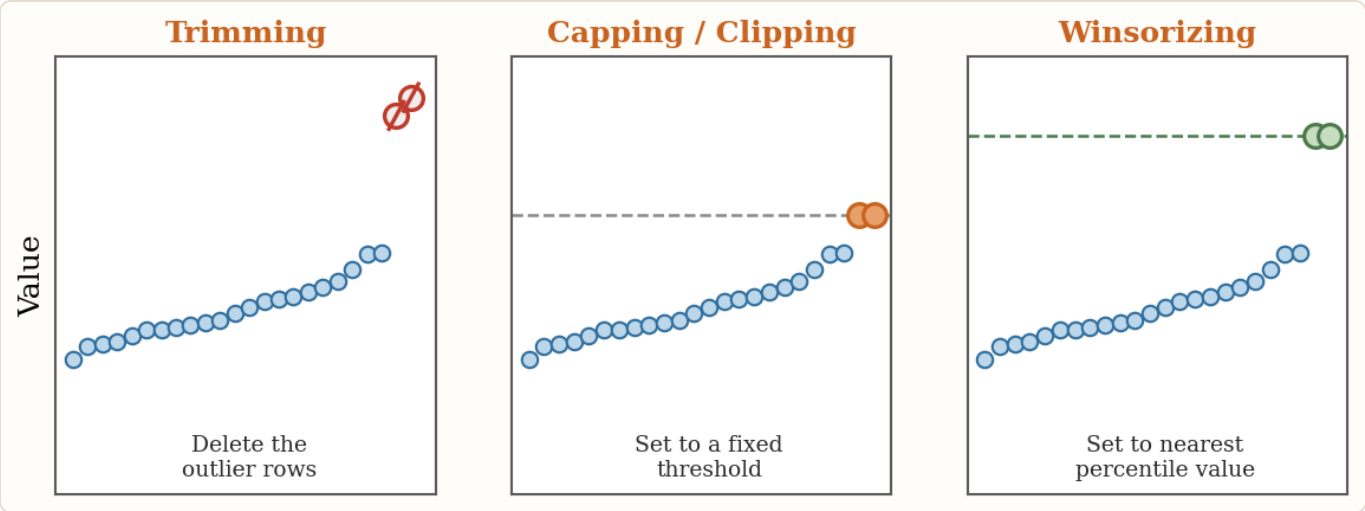
IQR vs. z-score on the same data. The IQR rule typically flags more points; the z-score rule is stricter and assumes a roughly normal shape.

WHICH ONE?

Use **IQR** as a robust default and for skewed data. Reach for **z-score** when the data is roughly normal, large, and has a meaningful mean and SD. The ±3 cutoff is a rule of thumb, not a law — adjust it if you have reason to.

11 Treating Outliers: Cap, Winsorize, Trim, Transform

Once found, you have two broad choices: **remove/limit** the outliers or **transform** the data to soften them.



Three ways to limit outliers. Trimming deletes the rows; capping/clipping sets them to a fixed threshold; winsorizing replaces them with the nearest percentile value from the data.

Removing or limiting

- **Trimming** — delete the rows containing outliers. Best when they're clearly errors.
- **Capping / clipping** — replace values beyond a *fixed* threshold with that threshold (e.g. everything above 1000 becomes 1000). The limit is chosen independently of the data.
- **Winsorization** — replace extremes with the nearest *percentile* value from the data (e.g. floor/cap at the 5th and 95th percentiles). Reduces their influence while keeping the row.

Transforming

- **Logarithmic transformation** — a log (often `log(1+x)`) compresses large values and spreads small ones; great for right-skewed data.
- **Square-root / Box-Cox / power transforms** — stabilize variance and make data more normal (Box-Cox needs positive values and a λ parameter).
- **Robust scaling** — subtract the median and divide by the IQR; like a z-score but resistant to outliers.

```
# Capping with the IQR bounds (the box-plot rule)
data.loc[data[col] < lower_bound, col] = lower_bound
data.loc[data[col] > upper_bound, col] = upper_bound
```

CAPPING VS. WINSORIZING

Both limit extremes, but **capping** uses a value chosen independently of the dataset, while **winsorizing** uses a value drawn *from* the data (a percentile). Trimming, by contrast, removes the rows entirely.

12 Study Recap: Key Q&A Concepts

These are the exact points most likely to trip you up on a quiz — distilled into quick, correct answers.

Question	Answer & why
Which technique uses **standard deviation** to find outliers?	**Z-score analysis.** It counts how many SDs a point is from the mean. (IQR uses quartiles, not SD.)
What does a z-score of **-3** mean?	The point is **three standard deviations below the mean** (negative = below).
Which methods **detect & cap** outliers?	The **Z-score** and **IQR** methods — each gives thresholds you can cap against.
Which technique replaces extremes with the **nearest in-range value**?	**Winsorization** — it replaces extreme values with the nearest in-range (percentile) value. This is distinct from **trimming**, which *removes* the outlier rows entirely. (Some quiz banks mis-key this to trimming, but winsorization is the concept the wording describes.)
Main disadvantage of `dropna`?	It **shrinks the dataset** and may drop rows with systematic missing patterns, losing information and adding bias.
Purpose of the **iterative imputer**?	To fill missing values by **modeling each feature as a function of the other features**.
When is **deletion** most appropriate?	When the proportion of nulls is **small** and the data is **MCAR**.
Purpose of the **simple imputer**?	To fill missing values using a single-column strategy — typically the **mean or median** (or mode/constant).
Why remove outliers before training?	They can **distort statistical analyses and model performance**, leading to inaccurate results (though some carry real signal).
Two steps **before** model training?	**Data preprocessing** and **feature engineering** (validation and tuning come later).
Tool to identify & visualize outliers?	The **box plot** — it shows quartiles, whiskers, and outlier points at a glance.

QR Quick Reference

Task	Tool / method	Key idea
Count missing values	`df.isnull().sum()`	Nulls per column.

Inspect data types & nulls	`df.info()`	Non-null counts + dtypes.
Summary statistics	`df.describe().T`	count, mean, std, min, quartiles, max.
Find duplicates	`df.duplicated().sum()`	Count exact repeat rows.
Drop missing rows / cols	`df.dropna() / dropna(axis=1)`	Simple but shrinks data.
Fill (single column)	`SimpleImputer`	mean / median / most_frequent / constant.
Fill (from other columns)	`IterativeImputer`	Multivariate, round-robin regression.
Visualize outliers	`sns.boxplot / plt.boxplot`	Points beyond whiskers = outliers.
IQR outlier bounds	`Q1-1.5*IQR, Q3+1.5*IQR`	Distribution-free; robust default.
Z-score outlier rule	`|(x-mean)/std| > 3`	Best for normal, large data.
Limit extremes	`clip / winsorize / trim`	Cap, percentile-cap, or delete.
Split before cleaning	`train_test_split`	Avoids data leakage.

G Glossary of Key Terms

The essential vocabulary from this guide, in plain language. (Your companion concept list contains the full 280-term set.)

Term	Definition
`Anomaly / Outlier`	A data point that differs markedly from the rest of the data; may be a rare true value or an error.
`Backward fill`	Filling a missing value with the next valid value that comes after it in sequence.
`Bias`	A systematic error that skews data, analysis, or predictions in one direction.
`Box plot`	A chart of the median, quartiles (Q1-Q3 box) and 1.5xIQR whiskers; points beyond the whiskers are drawn as outliers.
`Box-Cox transformation`	A family of power transforms that stabilize variance and improve normality (needs positive values, uses a lambda parameter).
`Capping / Clipping`	Replacing values beyond a fixed threshold with that threshold value.
`Collective outlier`	A group of points that is unusual together even if each point looks normal alone.
`Contextual outlier`	A value unusual only under a specific condition, time, or setting.
`Data leakage`	When information from the test set influences cleaning or training, inflating results. Split first to avoid it.
`Data preprocessing`	Cleaning and transforming raw data into a model-ready format; model-agnostic.

`dropna`	Pandas method that removes rows (or columns, axis=1) containing missing values.
`Explicit / Implicit null`	Explicit = formally marked (NaN, NULL, None). Implicit = unmarked, e.g. empty string, whitespace, placeholder.
`Feature engineering`	Creating or reshaping input features to help a specific model; uses domain knowledge.
`fit / transform / fit_transform`	fit learns from data; transform applies it; fit_transform does both at once.
`Forward fill`	Filling a missing value with the last valid value that came before it.
`Global outlier`	A point extreme relative to the entire dataset.
`Grubbs' test`	A statistical test for detecting a single outlier in normally distributed data.
`Imputation`	Replacing missing values with estimated substitutes instead of deleting them.
`Interquartile range (IQR)`	Q3 - Q1: the spread of the middle 50% of the data; basis of the box-plot outlier rule.
`Isolation forest`	An algorithm that detects outliers by random partitioning; outliers isolate in fewer splits.
`IterativeImputer`	Scikit-learn multivariate imputer that models each feature from the others in repeated rounds.
`MCAR / MAR / MNAR`	Missing Completely At Random / At Random (tied to observed data) / Not At Random (tied to the missing value itself).
`Mean / Median / Mode imputation`	Filling nulls with a column's mean or median (numeric) or mode (categorical).
`Null value`	The absence of data; not a zero and not an empty string.
`Outlier treatment`	Handling outliers by removing, capping, or transforming them.
`Q1 / Q3`	The 25th and 75th percentiles; the lower and upper edges of a box plot's box.
`Robust scaling`	Subtract the median and divide by the IQR; an outlier-resistant alternative to z-scoring.
`SimpleImputer`	Scikit-learn univariate imputer using a single-column strategy (mean, median, most_frequent, constant).
`Standard deviation`	A measure of how spread out values are around the mean.
`Trimming`	Removing the rows that contain outlier values.
`Winsorization`	Replacing extreme outliers with the nearest in-range percentile value from the data.
`Z-score`	

The number of standard deviations a value is from the mean; |z| > 3 is a common outlier rule.

L Learn More

A beginner-friendly path for learning to inspect and clean tabular data. Work through resources 1–5 first, clean one real CSV with pandas, then move to 7–9 for ML-safe preprocessing.

1. Kaggle Learn — Data Cleaning
`https://www.kaggle.com/learn/data-cleaning`
Best starting point: a free, hands-on course covering missing values, normalization, dates, encodings, and inconsistent entries.

2. pandas — 10 Minutes to pandas
`https://pandas.pydata.org/docs/user_guide/10min.html`
The essential Python operations for loading, inspecting, filtering, and modifying data.

3. pandas — Working with Missing Data
`https://pandas.pydata.org/docs/user_guide/missing_data.html`
Practical reference for detecting, removing, filling, and interpolating missing values.

4. pandas — Indexing & Duplicate Data
`https://pandas.pydata.org/docs/user_guide/indexing.html`
How to locate and remove duplicate rows using duplicated() and drop_duplicates().

5. Google ML Crash Course — Scrubbing
`https://developers.google.com/machine-learning/crash-course/numerical-data/scrubbing`
Beginner-friendly explanation of checking for missing values, duplicates, invalid ranges, and bad labels.

6. Google — Data Quality & Interpretation
`https://developers.google.com/machine-learning/guides/data-traps/quality`
Why dirty, biased, or poorly interpreted data can undermine an ML model.

7. scikit-learn — Imputation of Missing Values
`https://scikit-learn.org/stable/modules/impute.html`
Examples using SimpleImputer, KNNImputer, missing-value indicators, and pipelines.

8. scikit-learn — Outlier & Novelty Detection
`https://scikit-learn.org/stable/modules/outlier_detection.html`
Isolation Forest, Local Outlier Factor, One-Class SVM, and outlier vs. novelty detection.

9. scikit-learn — Common Pitfalls
`https://scikit-learn.org/stable/common_pitfalls.html`

Crucial for avoiding data leakage when cleaning, imputing, scaling, and transforming.

10. OpenRefine — Official User Manual
`https://openrefine.org/docs.html`

A visual, low-code option for exploring and cleaning CSV or spreadsheet data.

> **CRUCIAL RULE**
>
> Split the data into training and test sets **before** learning any imputation, scaling, or outlier-removal rule.
